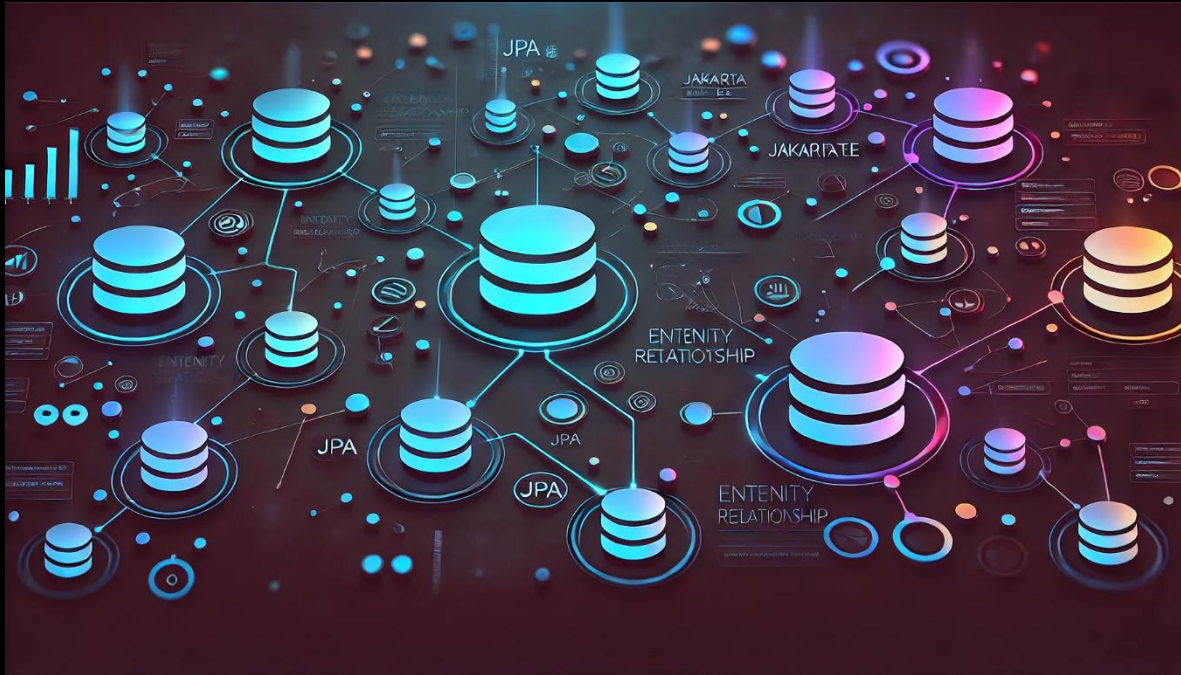


Asociaciones Lazy e Eager con JPA



Asociaciones Lazy e Eager con JPA

Uso de Asociaciones y Fetch Type en JPA con Jakarta EE

En este ejercicio aprenderemos cómo manejar **asociaciones en JPA** y cómo afectan el rendimiento de las consultas a la base de datos. Exploraremos los modos **Lazy (Carga Diferida)** y **Eager (Carga Anticipada)** para definir el comportamiento de las relaciones entre entidades.

1 Implementación de Asociaciones en JPA

A continuación, presentamos el código actualizado de la clase `ClienteAsociacionesJPA`, el cual permite recuperar entidades relacionadas y definir cómo se cargan.

Código de `ClienteAsociacionesJPA.java` (Actualizado)

```
package sga.cliente.asociaciones;
```

```

import java.util.List;
import jakarta.persistence.*;
import sga.domain.Persona;
import sga.domain.Usuario;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class ClienteAsociacionesJPA {

    private static final Logger log =
        LoggerFactory.getLogger(ClienteAsociacionesJPA.class);

    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");

        try (EntityManager em = emf.createEntityManager()) {
            // Recuperar todas las personas
            List<Persona> personas =
                em.createNamedQuery("Persona.findAll", Persona.class).getResultList();

            // Imprimir los objetos de tipo persona y sus usuarios asociados
            for (Persona persona : personas) {
                log.info("Persona: {}", persona);
                for (Usuario usuario : persona.getUsuarioList()) {
                    log.info("Usuario asociado: {}", usuario);
                }
            }
        }
    }
}

```

✓ Ejecución y Verificación (Modo Lazy)

Ejecutamos para comprobar que el modo por default para las asociaciones es Lazy Loading (carga perezosa) ya que verás múltiples consultas **separadas** para usuarios.

1. **Ejecutar la clase `ClienteAsociacionesJPA`.**
2. **Observar la salida en los logs**, donde se imprimirán las personas y sus usuarios asociados.

```
[EL Fine]: sql: ServerSession(1692885405)--Connection(553672556)--SELECT id_persona,
APELLIDO, EMAIL, NOMBRE, TELEFONO FROM PERSONA
```

```
[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Persona: Persona{idPersona=1,
nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}
```

```
[EL Fine]: sql: ServerSession(1692885405)--Connection(553672556)--SELECT id_usuario,
PASSWORD, USERNAME, id_persona FROM USUARIO WHERE (id_persona = ?)

bind => [1]

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Usuario asociado:
Usuario{idUsuario=1, username=rguzman, password=123, persona=Persona{idPersona=1,
nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}}

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Persona: Persona{idPersona=2,
nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}

[EL Fine]: sql: ServerSession(1692885405)--Connection(553672556)--SELECT id_usuario,
PASSWORD, USERNAME, id_persona FROM USUARIO WHERE (id_persona = ?)

bind => [2]

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Usuario asociado:
Usuario{idUsuario=2, username=ilara, password=123, persona=Persona{idPersona=2,
nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}}

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Persona: Persona{idPersona=3,
nombre=Maria, apellido=Esparza, email=mesparza@mail.com, telefono=55112233}

[EL Fine]: sql: ServerSession(1692885405)--Connection(553672556)--SELECT id_usuario,
PASSWORD, USERNAME, id_persona FROM USUARIO WHERE (id_persona = ?)

bind => [3]
```

2 Configuración de Lazy y Eager Loading en JPA

JPA permite definir cómo se cargan las relaciones entre entidades mediante la anotación `fetch = FetchType.LAZY` (Carga Diferida, este es el comportamiento por default como ya comprobamos si es que no se especifica otra cosa) o `fetch = FetchType.EAGER` (Carga Inmediata).

Código de `Persona.java` (Actualizado)

```
package sga.domain;

import jakarta.persistence.*;
import jakarta.validation.constraints.Size;
import java.io.Serializable;
import java.util.List;

@Entity
@NamedQueries({
    @NamedQuery(name = "Persona.findAll", query = "SELECT p FROM Persona p"),
    @NamedQuery(name = "Persona.findByIdPersona",
```

```

        query = "SELECT p FROM Persona p WHERE p.idPersona = :idPersona"),
    @NamedQuery(name = "Persona.findByNombre",
        query = "SELECT p FROM Persona p WHERE p.nombre = :nombre"),
    @NamedQuery(name = "Persona.findByApellido",
        query = "SELECT p FROM Persona p WHERE p.apellido = :apellido"),
    @NamedQuery(name = "Persona.findByEmail",
        query = "SELECT p FROM Persona p WHERE p.email = :email"),
    @NamedQuery(name = "Persona.findByTelefono",
        query = "SELECT p FROM Persona p WHERE p.telefono = :telefono"))}

public class Persona implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "id_persona")
    private Integer idPersona;
    @Size(max = 45)
    private String nombre;
    @Size(max = 45)
    private String apellido;
    // @Pattern(regexp="[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\\.[a-z0-9!#$%&'*/+=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?", message="Invalid email")//if the
    // field contains email address consider using this annotation to enforce field validation
    @Size(max = 45)
    private String email;
    @Size(max = 45)
    private String telefono;
    @OneToMany(mappedBy = "persona", fetch = FetchType.EAGER)
    private List<Usuario> usuarioList;

    public Persona() {
    }

    public Persona(String nombre, String apellido, String email, String telefono) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
    }

    public Persona(Integer idPersona) {
        this.idPersona = idPersona;
    }
}

```

```
public Integer getIdPersona() {  
    return idPersona;  
}  
  
public void setIdPersona(Integer idPersona) {  
    this.idPersona = idPersona;  
}  
  
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public String getApellido() {  
    return apellido;  
}  
  
public void setApellido(String apellido) {  
    this.apellido = apellido;  
}  
  
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public String getTelefono() {  
    return telefono;  
}  
  
public void setTelefono(String telefono) {  
    this.telefono = telefono;  
}  
  
public List<Usuario> getUsuarioList() {  
    return usuarioList;  
}  
  
public void setUsuarioList(List<Usuario> usuarioList) {
```

```

        this.usuarioList = usuarioList;
    }

    @Override
    public int hashCode() {
        int hash = 0;
        hash += (idPersona != null ? idPersona.hashCode() : 0);
        return hash;
    }

    @Override
    public boolean equals(Object object) {
        // TODO: Warning - this method won't work in the case the id fields are not set
        if (!(object instanceof Persona)) {
            return false;
        }
        Persona other = (Persona) object;
        if ((this.idPersona == null && other.idPersona != null) || (this.idPersona != null &&
!this.idPersona.equals(other.idPersona))) {
            return false;
        }
        return true;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" +
apellido + ", email=" + email + ", telefono=" + telefono + '}';
    }
}

```

✖ Explicación (Modo Lazy, comportamiento por default):

- @OneToMany(mappedBy = "persona", fetch = FetchType.LAZY):
 - **Carga diferida (LAZY):** los usuarios asociados **no se recuperan de inmediato** con la persona.
 - Se cargan **solo cuando se accede a persona.getUsuarioList()**.
- @OneToMany(mappedBy = "persona", fetch = FetchType.EAGER):
 - **Carga anticipada (EAGER):** cada vez que se recupera una persona, **sus usuarios relacionados también se cargan inmediatamente**.

Código de Usuario.java (Actualizado)

```
package sga.domain;

import jakarta.persistence.*;
import jakarta.validation.constraints.Size;
import java.io.Serializable;

@Entity
@NamedQueries({
    @NamedQuery(name = "Usuario.findAll", query = "SELECT u FROM Usuario u"),
    @NamedQuery(name = "Usuario.findByIdUsuario",
        query = "SELECT u FROM Usuario u WHERE u.idUsuario = :idUsuario"),
    @NamedQuery(name = "Usuario.findByUsername",
        query = "SELECT u FROM Usuario u WHERE u.username = :username"),
    @NamedQuery(name = "Usuario.findByPassword",
        query = "SELECT u FROM Usuario u WHERE u.password = :password"))})
public class Usuario implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "id_usuario")
    private Integer idUsuario;

    @Size(max = 45)
    private String username;
    @Size(max = 45)
    private String password;

    @JoinColumn(name = "id_persona", referencedColumnName = "id_persona")
    @ManyToOne(fetch = FetchType.EAGER)
    private Persona persona;

    public Usuario() {
    }

    public Usuario(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public Usuario(Integer idUsuario) {
        this.idUsuario = idUsuario;
    }
}
```

```
}

public Integer getIdUsuario() {
    return idUsuario;
}

public void setIdUsuario(Integer idUsuario) {
    this.idUsuario = idUsuario;
}

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public Persona getPersona() {
    return persona;
}

public void setPersona(Persona persona) {
    this.persona = persona;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (idUsuario != null ? idUsuario.hashCode() : 0);
    return hash;
}

@Override
public boolean equals(Object object) {
    // TODO: Warning - this method won't work in the case the id fields are not set
    if (!(object instanceof Usuario)) {
```

```

        return false;
    }
    Usuario other = (Usuario) object;
    if ((this.idUsuario == null && other.idUsuario != null) || (this.idUsuario != null &&
!this.idUsuario.equals(other.idUsuario))) {
        return false;
    }
    return true;
}

@Override
public String toString() {
    return "Usuario{" + "idUsuario=" + idUsuario + ", username=" + username + ", password=" +
password + ", persona=" + persona + '}';
}
}

```

✚ Explicación:

- @ManyToOne(fetch = FetchType.LAZY):
 - **Carga diferida (LAZY):** si se recupera un usuario, no se recupera su persona asociada.
- @ManyToOne(fetch = FetchType.EAGER):
 - **Carga anticipada (EAGER):** cada vez que se recupera un usuario, **su persona relacionada también se carga inmediatamente (comportamiento por default en este tipo de asociaciones).**

✓ Ejecución y Verificación (Modo Eager)

Ejecutamos para comprobar el modo Eager para las asociaciones de objetos persona y sus usuarios asociados. Observamos que desde el inicio se cargan las personas y sus usuarios asociados, sin esperar a llamar al método `persona.getUsuarioList()`.

1. Hacer Clean & Build para obtener la última versión del proyecto
2. **Ejecutar la clase ClienteAsociacionesJPA.**
3. **Observar la salida en los logs,** donde se imprimirán las personas y sus usuarios asociados.

```
[EL Fine]: sql: ServerSession(1923999715)--Connection(1542598424)--SELECT id_persona,
APELLIDO, EMAIL, NOMBRE, TELEFONO FROM PERSONA
```

```
[EL Fine]: sql: ServerSession(1923999715)--Connection(1542598424)--SELECT id_usuario,
PASSWORD, USERNAME, id_persona FROM USUARIO WHERE (id_persona = ?)
```

```
bind => [1]
```

```
[EL Fine]: sql: ServerSession(1923999715)--Connection(1542598424)--SELECT id_usuario,
PASSWORD, USERNAME, id_persona FROM USUARIO WHERE (id_persona = ?)

bind => [2]

[EL Fine]: sql: ServerSession(1923999715)--Connection(1542598424)--SELECT id_usuario,
PASSWORD, USERNAME, id_persona FROM USUARIO WHERE (id_persona = ?)

bind => [3]

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Persona: Persona{idPersona=1,
nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Usuario asociado:
Usuario{idUsuario=1, username=rguzman, password=123, persona=Persona{idPersona=1,
nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}}

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Persona: Persona{idPersona=2,
nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Usuario asociado:
Usuario{idUsuario=2, username=ilara, password=123, persona=Persona{idPersona=2,
nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}}

[main] INFO sga.cliente.asociaciones.ClienteAsociacionesJPA - Persona: Persona{idPersona=3,
nombre=Maria, apellido=Esparza, email=mesparza@mail.com, telefono=55112233}
```

3 Diferencia entre Lazy y Eager Loading

Modo de Carga	Descripción	Ventajas	Desventajas
Lazy	Carga diferida de la relación	Ahorra recursos si la relación no siempre se necesita	Puede generar múltiples consultas
Eager	Carga inmediata de la relación	Reduce el número de consultas	Puede traer datos innecesarios, impactando el rendimiento

4 Ejecución y Verificación

Para comprobar que la configuración de carga funciona correctamente:

1. **Ejecutar la clase `ClienteAsociacionesJPA`.**
2. **Observar la salida en los logs**, donde se imprimirán las personas y sus usuarios asociados.
3. **Comparar los logs generados:**
 - Si usaste `LAZY`, verás múltiples consultas **separadas** para usuarios.

- Si usaste **EAGER**, verás que los usuarios se cargaron en **una sola consulta** junto con la persona.
-

Conclusión

- **Lazy Loading (LAZY)** es útil para mejorar el rendimiento cuando no siempre se necesita la relación.
- **Eager Loading (EAGER)** carga todo de inmediato, reduciendo el número de consultas pero consumiendo más recursos.
- Es importante elegir el modo adecuado según las necesidades del proyecto.

 **¡Ahora sabes cómo manejar asociaciones y configurar el `FetchType` en JPA!**

Sigue adelante con tu aprendizaje  , ¡el esfuerzo vale la pena!

¡Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)